

Handover: trade_partners / trade_commodities / trade_total

Scope: the three UN Comtrade mart models in `cp_dbt`, their incremental behaviour, and the Airflow tasks around them.

Status: fixes applied; see "Open items" at the end for what still needs confirming.

1. What was wrong

1.1 The incremental models were never actually incremental

`cp_transformation_dag.py` has a `clean_transformed_tables` task that runs `DROP TABLE ... CASCADE` for every table in the `TRANSFORMED_TABLES` list, before dbt runs. `trade_partners` was still in that list.

Because the table was dropped just before every run, `is_incremental()` was always `False`, so dbt took the full-rebuild branch every single time. The model was configured as incremental but had never once behaved incrementally. That is also why the table was found empty after a failure: it started every run empty by design, and the failure meant nothing was put back.

`trade_total` and `trade_commodities` were already commented out of that list — someone had hit this before and excluded them, and `trade_partners` was missed.

1.2 `id` collided across batches

All three models computed `id` as `ROW_NUMBER() OVER (...)`. That numbers rows *within the current batch only*, restarting at 1 each run. Different rows from different runs therefore received the same `id`, which is why the primary key rebuild in the `post_hook` failed with duplicate-key errors.

1.3 The `post_hooks` were the bulk of the runtime

Each model's `post_hook` dropped and re-added the primary key and all foreign keys across the whole table, on every run — re-validating every existing row every time. `trade_commodities` additionally had `ADD CONSTRAINT ... PRIMARY KEY`

with `no DROP ... IF EXISTS` before it, so every run after the first errored on that line.

1.4 Year bounds were hardcoded

`process_from_year` and `current_year` were both pinned to `2024`, with `-- CHANGE THIS LINE` comments in the source. Every run silently ignored anything outside that one year.

1.5 The raw table had no usable index for the year filter

`raw.raw_un_comtrade` (~2B rows) was filtered with `"period"::int >= ...` and `"refYear"::int >= ...`. Those columns are text; a plain index on a text column cannot serve a *casted* comparison, so Postgres did a sequential scan of all 2B rows for every batch. This is why year-by-year batching alone did not help.

1.6 Duplicates

Two separate sources:

- **Overlapping DAG runs** — a second backfill was triggered while the first was still active, so both inserted the same rows.
- **Genuine duplicates in the raw source** — confirmed as exact duplicate rows.
`trade_commodities` already had `SELECT DISTINCT` (evidently patched around this before); the other two did not.

2. What was changed

2.1 `cp_transformation_dag.py` (yearly)

- `trade_partners` commented out of `TRANSFORMED_TABLES` so it is no longer dropped before each run. **This is the single most important change.**
- New `ensure_raw_indexes` task, running between `check_raw_schema` and `clean_transformed_tables`. It re-creates the two raw indexes with `IF NOT EXISTS` (a no-op when they exist) and runs `ANALYZE`. This covers the case where a raw reload drops and recreates the raw table, taking the indexes with it. It is placed before any drop happens, so a failure there costs nothing.
- `run_incremental_models` timeout reduced from 12h to 2h. Observed real runtime was 5–8 minutes per model.

2.2 The three model files

- **Strategy:** `materialized='incremental'`, `incremental_strategy='delete+insert'`, `unique_key='uid'`. Note `merge` is **not supported** by this dbt/adaptor version — it was tried and rejected at compile time with "the incremental strategy 'merge' is not valid for this adapter".
- **Year bounds:** now dynamic, driven by `run_started_at`, with three optional vars:
 - `backfill_mode + backfill_start_year + backfill_end_year` — forces an exact window, used by the backfill DAG so each task does exactly one year.
 - `as_of_year` — overrides "current year".
 - `rebuild_constraints` — see below.
- **post_hooks split in two:**
 - FK re-attach runs *every* time, using `NOT VALID` so it never scans the table. This is required because `countries` and `kpis` are dropped with `CASCADE` every run, which takes these FKs with them.
 - The `uid` UNIQUE constraint rebuild is gated behind `rebuild_constraints=true` — backfill only, never a normal run.
 - The comment text sits *inside* the `{% if %}` block. When it was outside, the hook rendered to comments-only and Postgres raised "can't execute an empty query".
- `SELECT DISTINCT` added where each raw source is read, in `trade_partners` and `trade_total`. `trade_commodities` already had it.
- `id` is no longer computed by the query. It is a column on the table with a `DEFAULT nextval(...)` from a standalone sequence. The final `SELECT` in each model does:

```
LEFT JOIN {{ this }} AS t ON t.uid = w.uid
...
COALESCE(t.id, nextval('<schema>.<table>_id_seq')) AS id
```

Do not remove this join. `delete+insert` deletes and re-inserts every row in the batch. Without the lookup, every reprocessed row would draw a fresh `id` each run. dbt builds the model's `SELECT` into a staging relation *before* deleting from the target, so the old ids are still readable when the join runs. The `is_incremental()` guard means a full refresh assigns fresh ids instead, which is correct there.

2.3 Keys

- `id` is the **PRIMARY KEY**, under the original constraint names (`top_countries_pkey`, `trade_commodities_pkey`, `trade_total_pkey`).
Note `trade_partners` ' constraints carry the old `top_countries_*` names because Postgres does not rename constraints when a table is renamed. These are the real names in the database — do not "correct" them.
- `uid` is now a plain **UNIQUE** constraint, not the PK. It is still what dbt matches rows on and still the guard against duplicates; `delete+insert` only needs a unique constraint, not a primary key.

2.4 Sequences

`id` sequences are standalone (`CREATE SEQUENCE`), deliberately **not** `GENERATED ALWAYS AS IDENTITY` and **not** `OWNED BY`. Both of those tie the sequence's lifetime to the column, so a `--full-refresh` would delete it and the next one would restart low, reissuing numbers already used by different rows. A standalone sequence survives that.

3. Files

Permanent — keep:

File	Role
<code>models/marts/trade_partners.sql</code>	model
<code>models/marts/trade_commodities.sql</code>	model
<code>models/marts/trade_total.sql</code>	model
<code>days/cp_transformation_dag.py</code>	yearly pipeline

One-time — retire once green:

File	Role
<code>macros/add_id_columns.sql</code> + <code>add_id_columns_dag.py</code>	creates sequences, adds <code>id</code> , backfills existing rows
<code>macros/promote_id_primary_key.sql</code> +	makes <code>id</code> the PK, demotes <code>uid</code> to

<code>promote_id_primary_key_dag.py</code>	UNIQUE
<code>backfill_dag.py</code>	year-by-year historical backfill, all three models in one trigger
<code>macros/create_raw_year_indexes.sql</code> + <code>create_raw_indexes_dag.py</code>	superseded by <code>ensure_raw_indexes</code> in the yearly DAG

Discard — produced early in the investigation, before the real files were available, never deployed: `partition_setup.sql`, `top_countries_fixed.sql`, `big_table_model.sql`.

4. Routine operation

A normal yearly run needs no intervention. It recomputes the window as `MAX(year) - 2` through the current year, reprocesses only those years, and leaves the rest of the history untouched. A newly published year is picked up automatically on the first run after it appears in the raw data.

`rebuild_constraints` is not passed by the yearly DAG, so it defaults to false and the expensive constraint work never runs on a routine schedule.

The `uid` UNIQUE constraint is the ongoing duplicate monitor. If a raw duplicate ever slips past `DISTINCT`, the run fails loudly with a duplicate-key error rather than writing bad data. No separate alerting is needed for that case.

Re-running a historical backfill

1. Deploy models and macros.
2. Trigger `trade_tables_historical_backfill` **once** — it loops the full year range internally via dynamic task mapping. Do not trigger per year.
`max_active_tasks=1` for the first run (the first task creates the table structure); raise it to 5–10 afterwards to parallelise.
3. `max_active_runs=1` prevents a second overlapping trigger — this was the cause of the earlier duplicates.

After a `--full-refresh`

`id` is not part of any model's SELECT, so a full refresh rebuilds the table without it. Re-run `add_id_columns` (and then `promote_id_primary_key`) afterwards. The sequences survive and continue from where they left off.

Useful checks

```
-- duplicates (expect zero rows)
SELECT uid, COUNT(*) FROM transformed_data.trade_partners
GROUP BY uid HAVING COUNT(*) > 1;

-- key layout: expect contype 'p' on id, 'u' on uid
SELECT conname, contype FROM pg_constraint
WHERE conrelid = 'transformed_data.trade_partners'::regclass;

-- raw indexes still present after a raw reload
SELECT indexname FROM pg_indexes WHERE tablename = 'raw_un_comtrade';
```

5. Open items

1. **Confirm `promote_id_primary_key` completed.** The first attempt failed ("index is already associated with a constraint" — `USING INDEX` cannot adopt a constraint-backed index). The macro was rewritten to build the PK fresh and is idempotent, but a successful run has not been confirmed. Check with the `pg_constraint` query above on all three tables.
2. **Confirm `id` is stable across a run.** Record a few `(uid, id)` pairs for recent years, run the pipeline, and check they are unchanged. This validates the `LEFT JOIN` behaviour; it has not been observed end to end.
3. **`DBT_PROJECT_DIR` in the one-time DAG templates is `/opt/airflow/dbt`,** but the real project path per `cp_transformation_dag.py` is `/opt/airflow/dbt_project/cp_dbt`. Fix before triggering any of them.
4. **`id` ordering semantics changed.** The old `row_number()` ordered by country/kpi/year within a batch; sequence values reflect insertion order only. That ordering was already meaningless across batches, but confirm nothing downstream relied on it.
5. **`country_id` / `kpi_id` come from `countries.id` / `kpis.id`,** which are rebuilt on every run. If those models reassign `id` values on rebuild, historical rows here could point at the wrong country/KPI through that integer column. The FKs are on the `_uid` columns, so referential integrity is unaffected. Worth verifying how `countries` and `kpis` assign `id`.
6. **`SELECT DISTINCT` in `trade_total` was added preventively,** not in response to a confirmed duplicate there. `trade_total`'s PK build succeeded across the full history, so its level rows are demonstrably clean — but

`total_trade_exchange` sums *before* the uid is computed, so a duplicate there would silently double a figure rather than raise an error. Keeping the `DISTINCT` is the cheap way to rule that out.